Universidad del Rosario | Escuela de Ciencias e Ingeniería | DEPE

INTRODUCCIÓN A PANDAS

SÁBADO
11 DE OCTUBRE
10:00 AM A 12:00 PM

Pandas es ampliamente utilizado en la ciencia de datos debido a su capacidad para cargar, limpiar y manipular datos de manera eficiente.

Taller Knuth

MODALIDAD ACCESO REMOTO

NO SE REQUIERE EXPERIENCIA PREVIA EN PROGRAMACIÓN

Sobre el conferencista



Miguel Angel Orjuela Rocha

Ing. de Sistemas y Computación

Skills:

- Desarrollo Fullstack
- Analítica de datos
- Transferencia de conocimiento

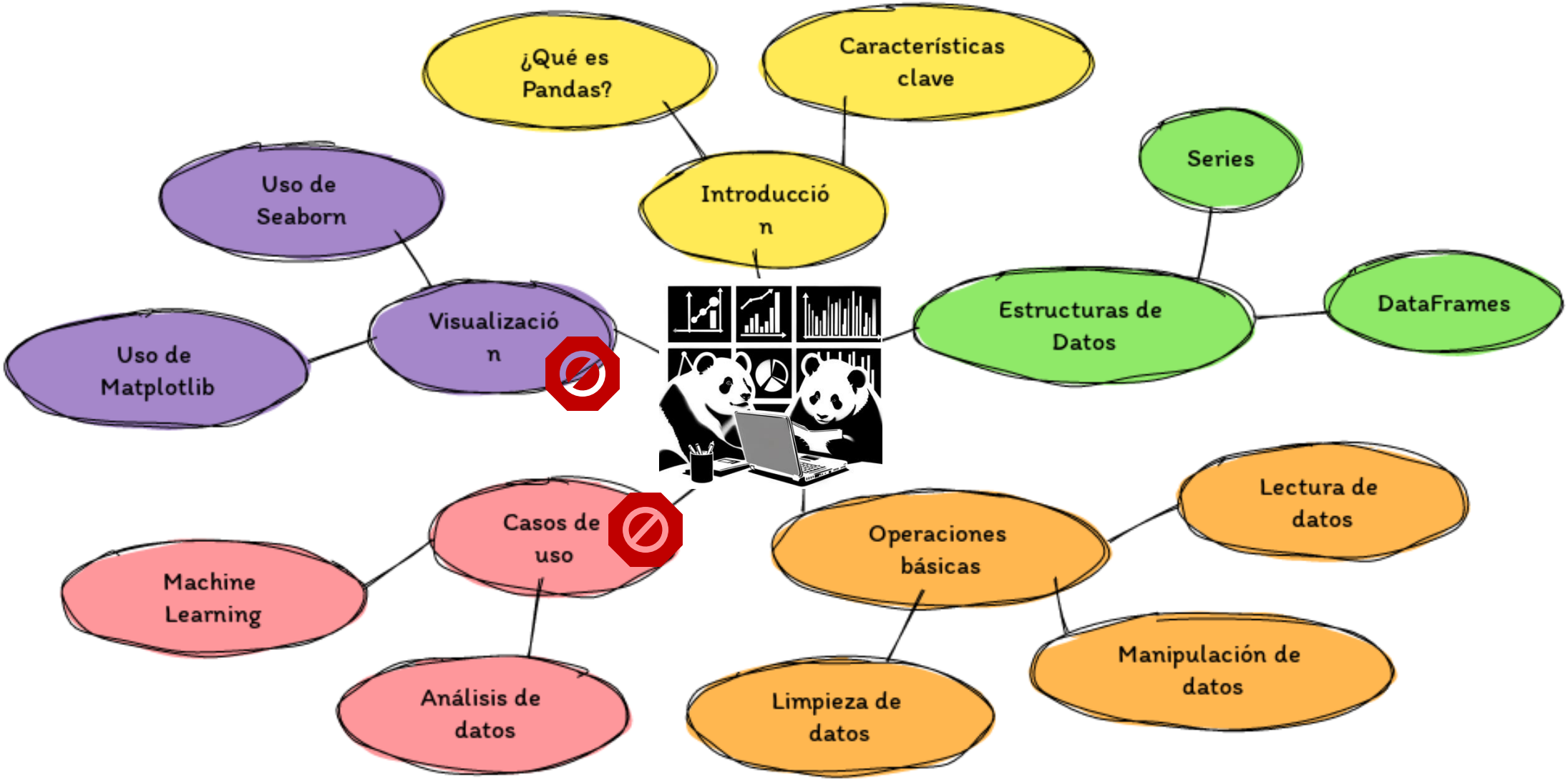
Contacto:

miguela.orjuela@urosario.edu.co

https://www.linkedin.com/in/miguel-orjuela/

https://github.com/maorjuela73




¿Qué es Pandas?
Características clave
Introducción
Uso de Seaborn
Visualización
Uso de Matplotlib
Series
Estructuras de Datos
DataFrames
Casos de uso
Machine Learning
Análisis de datos
Operaciones básicas
Lectura de datos
Manipulación de datos
Limpieza de datos

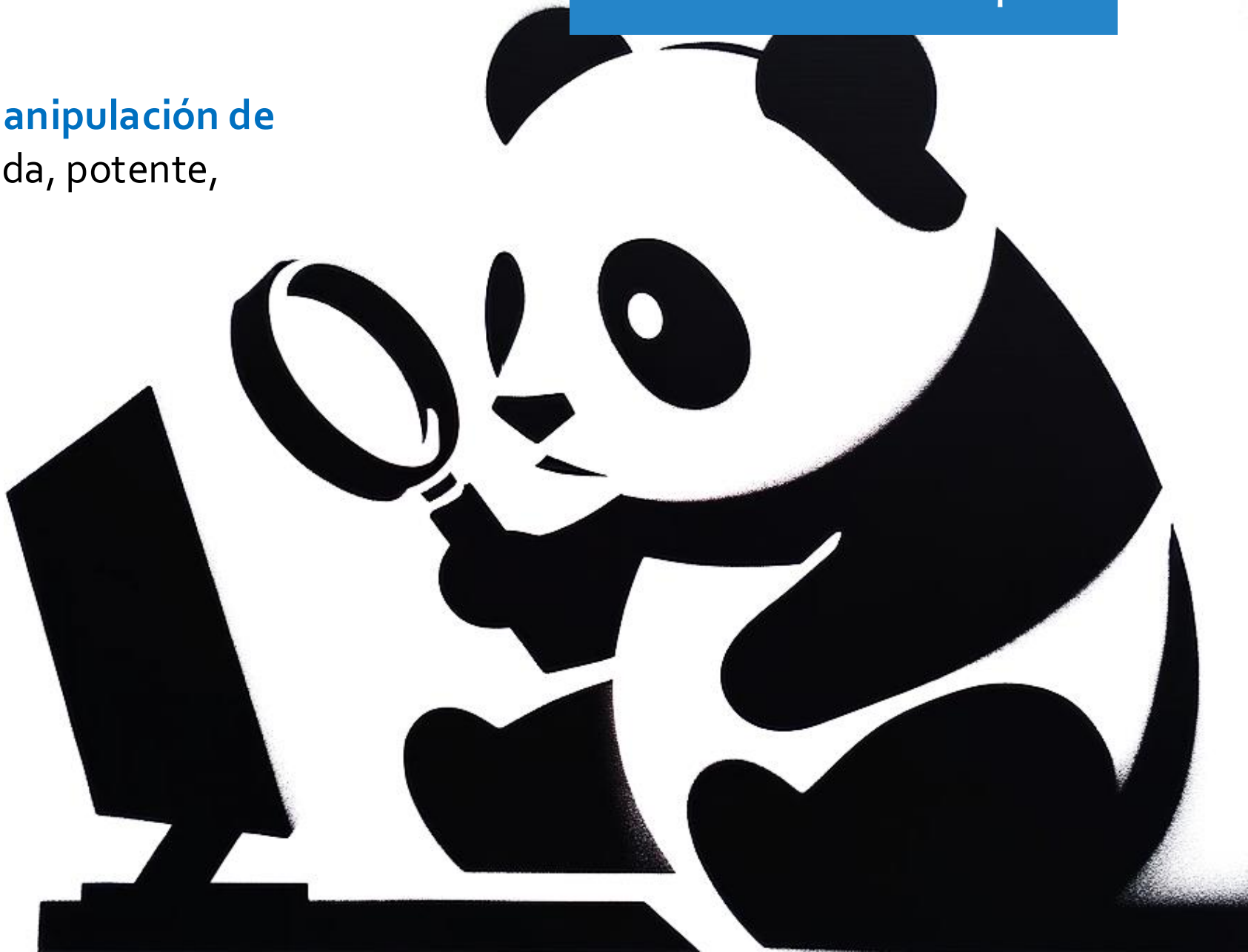
Introducción

¿Qué es pandas?

Herramienta de **análisis** y **manipulación de datos** de código abierto rápida, potente, flexible y fácil de usar.

pandas

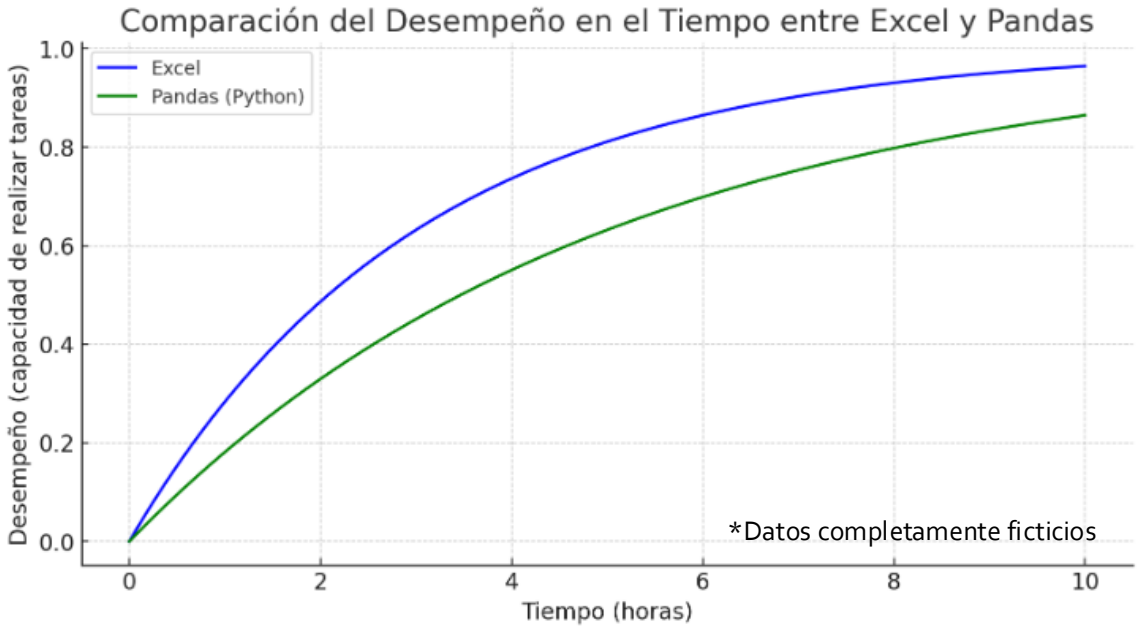
https://pandas.pydata.org/



Diferencias

Pandas vs. Excel

Criterio	Pandas	Excel
Cantidad de Interacciones	Alto número de comandos; menos clicks. Interacción a través de código en un entorno de programación.	Menor número de comandos; más clicks. La interacción es visual y directa con la interfaz gráfica.
Cantidad de Suposiciones	Requiere comprensión previa de estructuras de datos y métodos de pandas; muchas funciones asumen un nivel de conocimiento sobre manipulación de datos.	Menos suposiciones técnicas iniciales; diseñado para ser intuitivo y accesible para usuarios no técnicos.
Curva de Aprendizaje	Más empinada; requiere conocimientos de Python y conceptos de programación.	Más suave al principio; accesible para principiantes, pero puede volverse complejo al profundizar en funciones avanzadas.



Similitudes

Uso de funciones

- Identificar la función correcta
- Darle los inputs correctos y en orden

Ejemplos en Excel

- SUMIF
- VLOOKUP



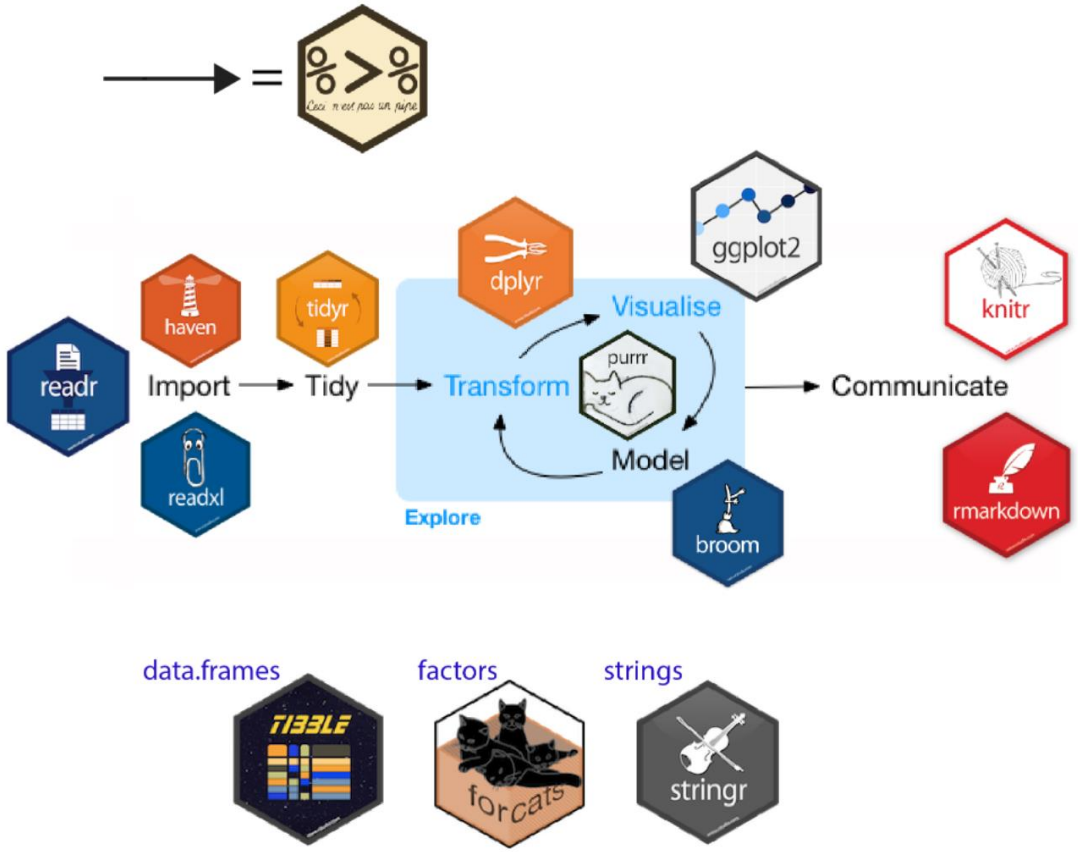
El poder de python + pandas

- Lenguaje de comunicación con el programa más verboso
- Jupyter o Colab
- Ecosistema de data science



Competidores de pandas

- RStudio y su tidyverse



Herramientas a usar (1)

Intérprete de Python

- https://www.python.org/

Entornos Integrados de Desarrollo (**IDEs**)

- https://www.spyder-ide.org/
- https://jupyter.org/try
- https://www.jetbrains.com/es-es/pycharm/
- https://code.visualstudio.com/
- https://www.sublimetext.com/
- …



Entornos Cloud

- https://colab.research.google.com/

Herramientas a usar (2)

Entorno de desarrollo

Librería “pandas”

Instalación e importación:

- pip install pandas (si es necesario)
- import pandas as pd

Configuración del entorno:

Uso de Jupyter Notebook, VSCode o IDEs compatibles.

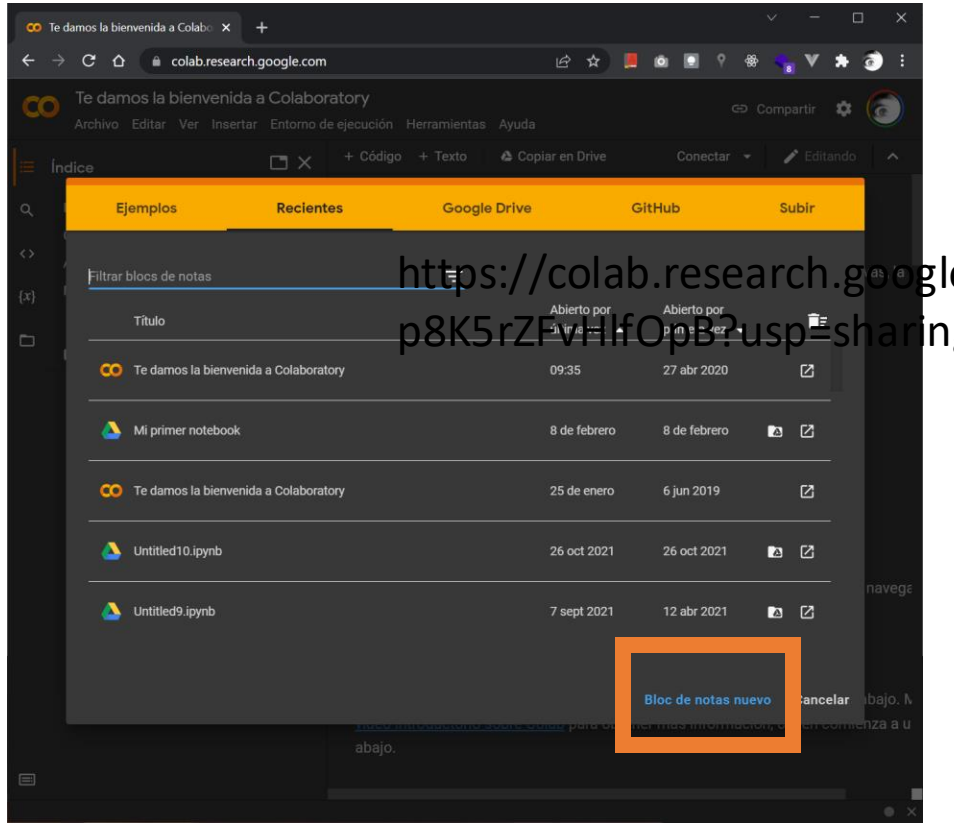
Consejo práctico:

Importar otras librerías complementarias (NumPy, Matplotlib) desde el inicio

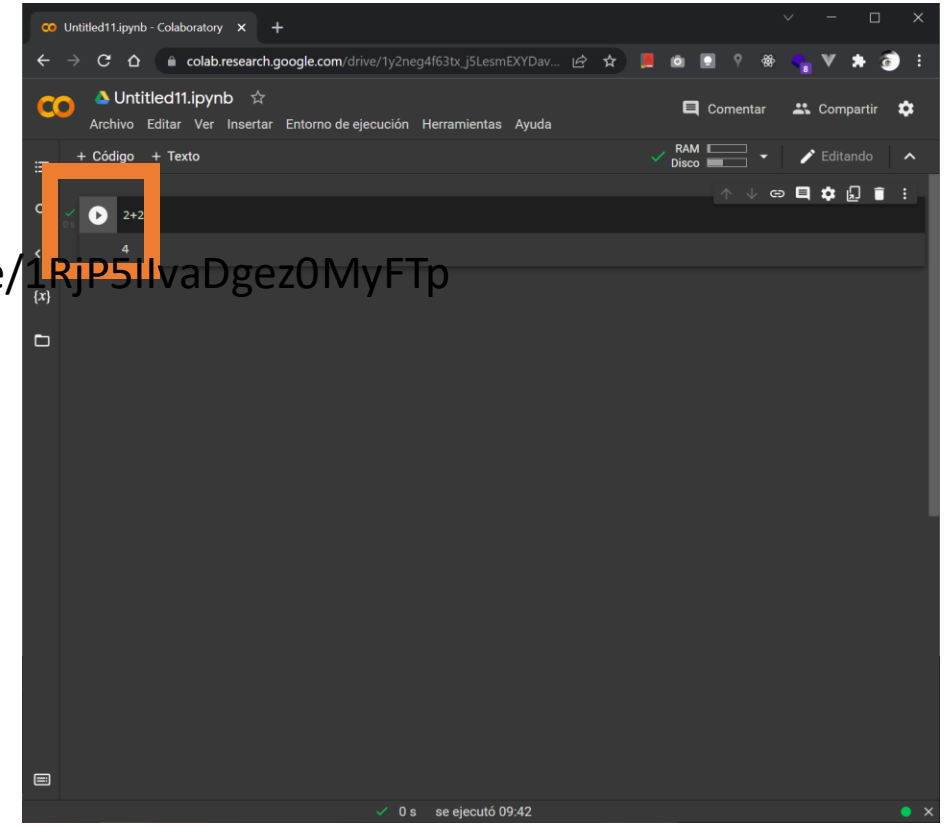


Ejecución de código en Google Colab

- Los cuadernos funcionan con celdas y ya **tienen pandas instalado.**
- Para ejecutar código de una celda se da clic en el botón Ejecutar (o Play).



https://colab.research.google.com/drive/1RjP5IlvaDgez0MyFTpp8K5rZFVHfOpB?usp=sharing



Notebook de actividades

Entorno de desarrollo

https://colab.research.google.com/drive/1RjP5IIvaDgezoMyFTpp8K5rZFvHlfOpB?usp=sharing



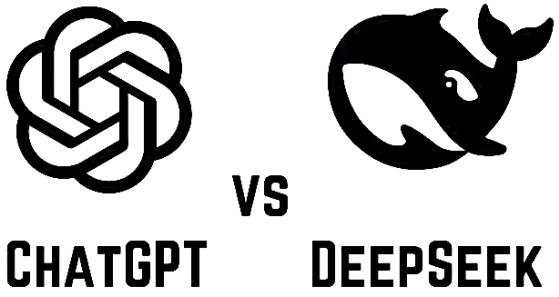
Conceptos básicos para entender pandas

Entorno de desarrollo

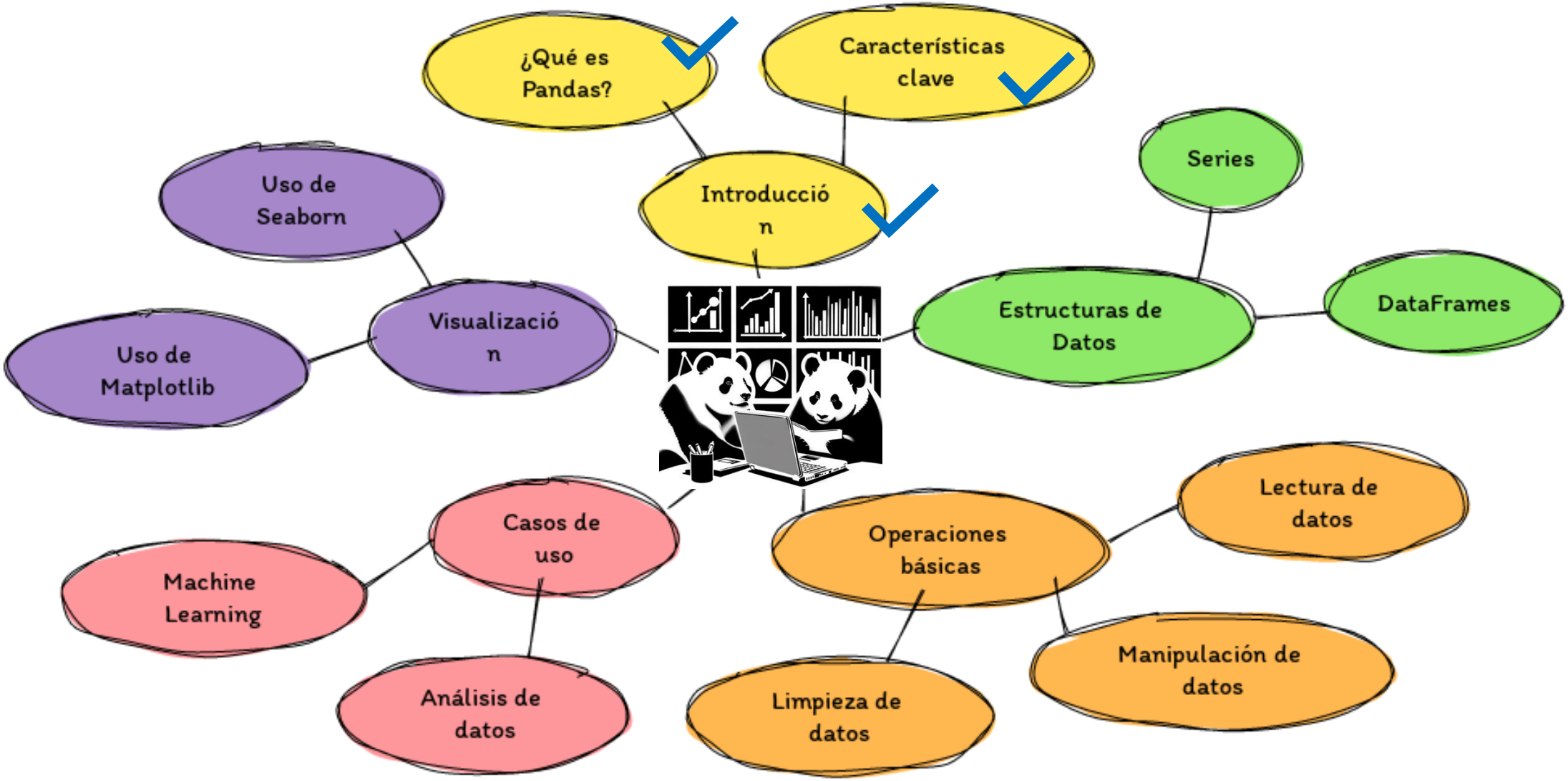
1. **Listas:** Estructuras de datos básicas para almacenar secuencias ordenadas.
2. **Diccionarios:** Almacenamiento de pares clave-valor para un acceso rápido y eficiente.
3. **Comprensión de Listas:** Técnica concisa para generar listas mediante operaciones en iterables.
4. **Funciones:** Encapsulación de lógica para reutilizar código y simplificar tareas.
5. **Manejo de Excepciones:** Control de errores que garantiza la robustez del programa.
6. **Importación de Módulos:** Integración de bibliotecas externas que extienden las capacidades de Python.
7. **Iteración:** Uso de bucles para recorrer y procesar colecciones de datos.
8. **Tipos de Datos Básicos:** Manejo de enteros, flotantes y cadenas, fundamentales para cualquier operación.

Fuentes de ayuda

- Documentación Oficial:
  - Sitios web y wikis de las librerías (ej. pandas.pydata.org, docs.scipy.org)
- Comunidades y Foros:
  - Stack Overflow
  - Grupos en Reddit y Slack
- Asistentes de programación



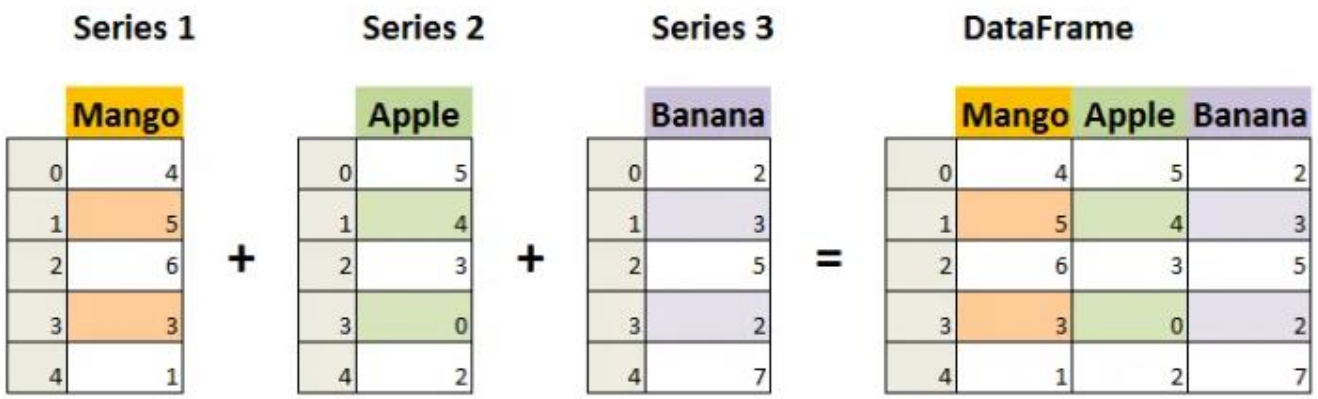



¿Qué es Pandas?
Características clave
Introducción
Uso de Seaborn
Visualización
Uso de Matplotlib
Series
Estructuras de Datos
DataFrames
Casos de uso
Machine Learning
Análisis de datos
Operaciones básicas
Lectura de datos
Manipulación de datos
Limpieza de datos

Estructuras de datos

El dataframe

- Estructura de datos principal en pandas.
- Es una tabla de datos con múltiples columnas (series).
- Una serie es una matriz etiquetada unidimensional, una sola columna de datos.



https://www.datasciencemadesimple.com/create-series-in-python-pandas/

Programar series


```python
import pandas as pd

# Serie de enteros
serie_enteros = pd.Series([10, 20, 30, 40, 50])

# Serie de flotantes
serie_flotantes = pd.Series([0.1, 0.2, 0.3, 0.4, 0.5])

# Serie de strings
serie_strings = pd.Series(["a", "b", "c", "d", "e"])

# Serie de booleanos
serie_booleanos = pd.Series([True, False, True, False, True])
```

Programar dataframes

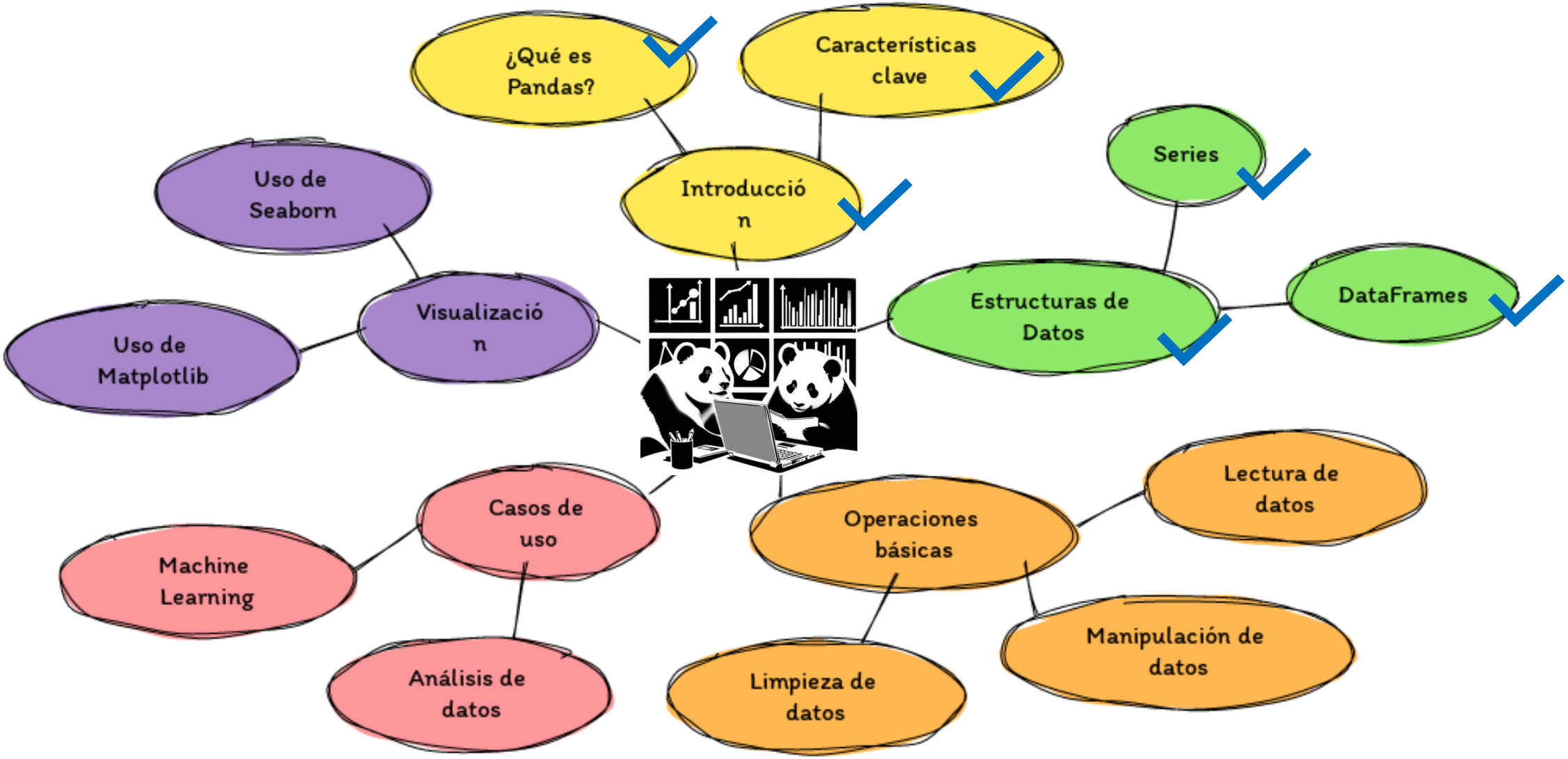
Pandas dataframe y series

```
# Crear DataFrame combinando las series
dataframe = pd.DataFrame({
    'Enteros': serie_enteros,
    'Flotantes': serie_flotantes,
    'Strings': serie_strings,
    'Booleanos': serie_booleanos
})
```

Actividad

- Crea una serie de 10 elementos, compuesta por números enteros aleatorios del 70 al 100, que representen las calificaciones de un examen mensual. Establece el índice de la serie con los nombres de los meses, comenzando en septiembre y terminando en junio.

- Revisa la creación del dataset de personas en formato Data Frame


¿Qué es Pandas?
Características clave
Introducción
Uso de Seaborn
Visualización
Uso de Matplotlib
Series
Estructuras de Datos
DataFrames
Casos de uso
Machine Learning
Análisis de datos
Operaciones básicas
Lectura de datos
Manipulación de datos
Limpieza de datos

Importación de un conjunto de datos

Carga de librería y datos

Importación de un conjunto de datos

- Datos a cargar

https://drive.google.com/file/d/18cigQy12DAqdwOd07iW0aiM9ZWE1JfUM/view?usp=sharing

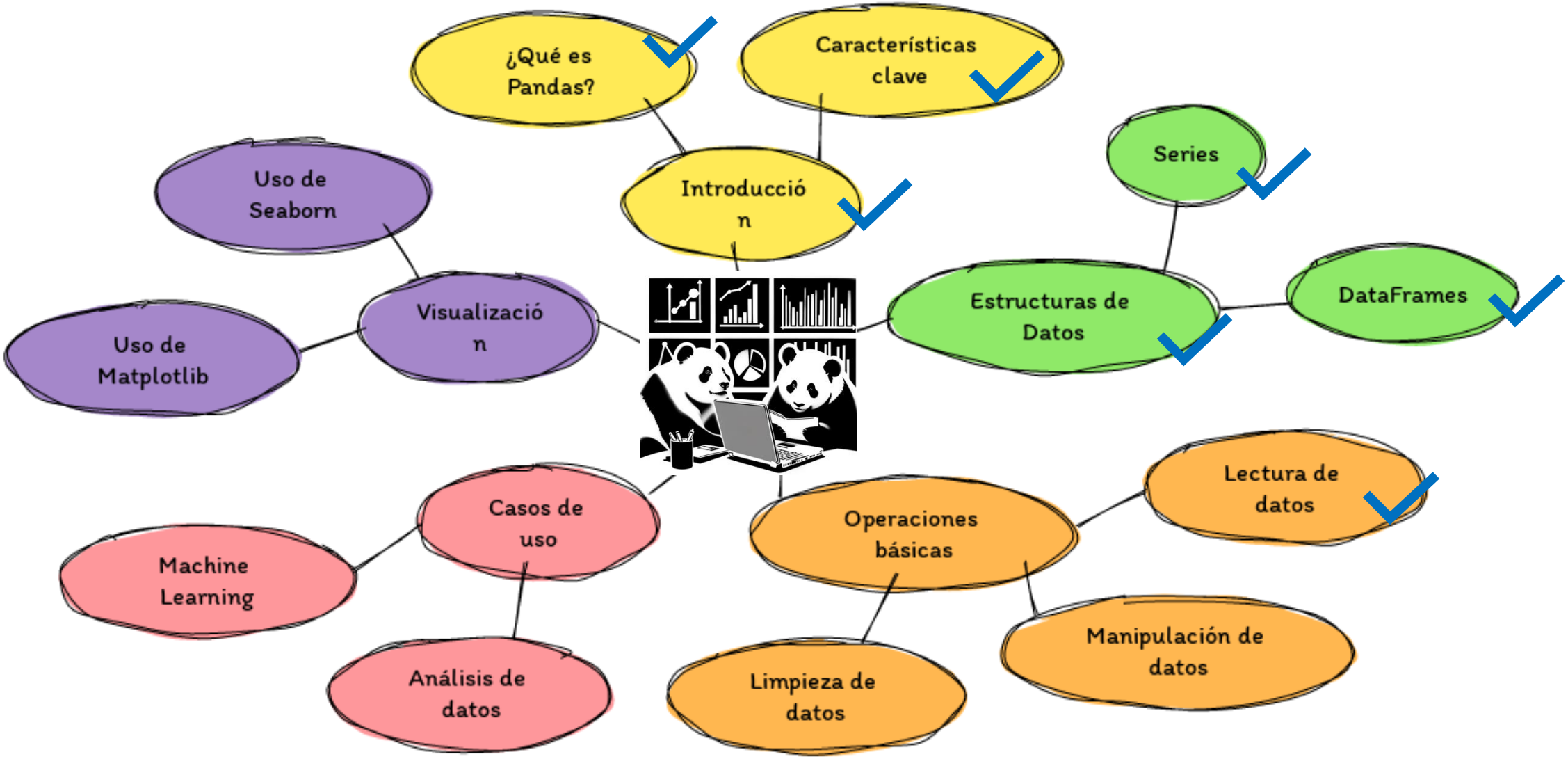
- Carga de la librería

```
import pandas as pd
```

- Lectura de un archivo

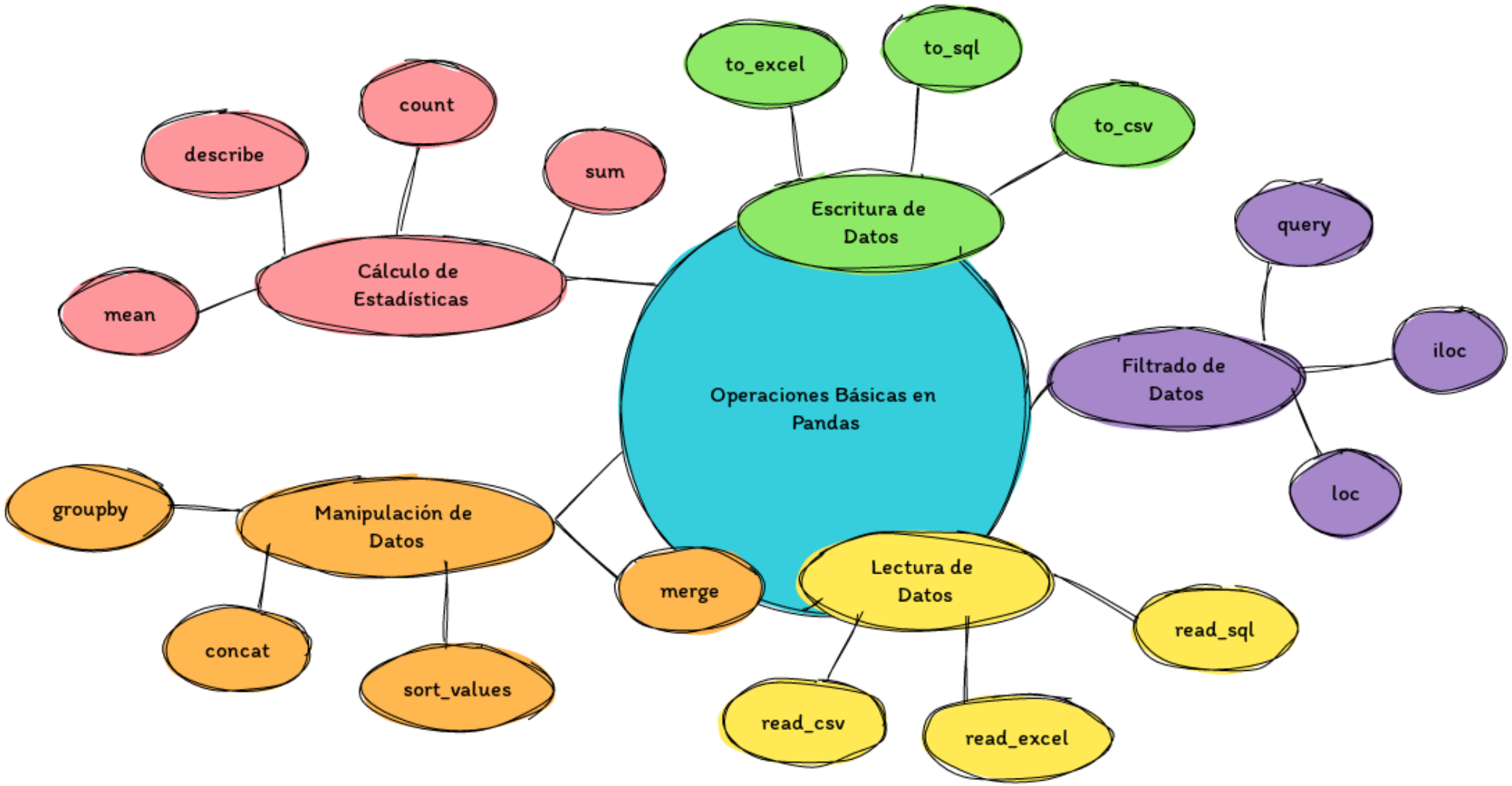
```
pd.read_csv("movies.csv")
```

```
pd.read_csv("movies.csv", index_col="Title")
```


¿Qué es Pandas?
Características clave
Introducción
Uso de Seaborn
Visualización
Uso de Matplotlib
Series
Estructuras de Datos
DataFrames
Casos de uso
Machine Learning
Análisis de datos
Operaciones básicas
Lectura de datos
Manipulación de datos
Limpieza de datos

Manipulación de dataframes

Pandas dataframe y series



Análisis básico

Información general del dataframe

- head y tail
- Número de filas, columnas y celdas
- Tipos de datos en cada columna

Acceder a una fila en una serie o dataframe

- Por su número de fila
- Por su etiqueta de índice

Ordenar datos

- Por una o varias columnas
- Por el índice



Análisis más sofisticado

Manipulación de dataframes

- Extraer columnas
  - Una
  - Varias
- Filtro por diferentes criterios
- Agrupación de datos

Análisis más sofisticado

Se pueden agrupar las filas del dataframe en función de los valores de una columna

Algunas operaciones de agregación sobre los grupos resultantes

- Conteos
- Sumas
- Promedios
- …

El objeto Series

¿Qué es una Serie?

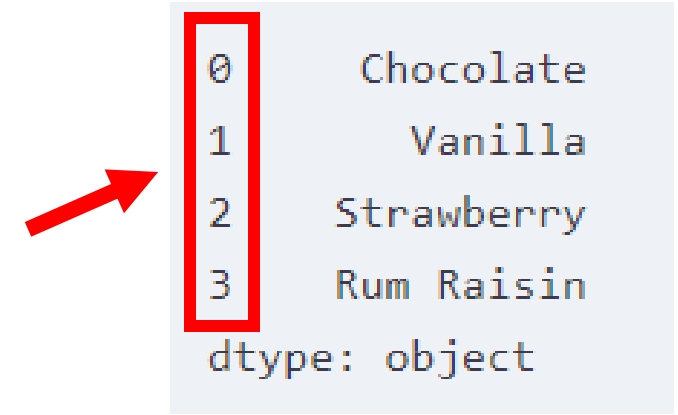
- Es un array unidimensional similar a una lista [ ]
- Tiene una etiqueta (label)
- Los datos son homogéneos
- Cada valor tiene una posición y una etiqueta que permite identificar cada valor (como un diccionario)
- Agrega más de 180 métodos para manipulación de datos

Instanciar una serie

El constructor recibe como primer parámetro (data) una lista de elementos

- Se puede asignar a cada valor de serie una etiqueta de índice.
- Las etiquetas de índice pueden ser de cualquier tipo de datos inmutables: cadenas, tuplas, fechas y horas y más.
- Se puede referenciar un valor por su orden o por una clave/etiqueta. Cada valor tiene dos identificadores.

Índice



Modificar el índice

El constructor recibe como segundo parámetro un índice personalizado

Índice

```
pd.Series(data = ice_cream_flavors, index = days_of_week)

Out [5] Monday          Chocolate
        Wednesday         Vanilla
        Friday         Strawberry
        Saturday       Rum Raisin
        dtype: object
```

- A diferencia de los diccionarios, el índice permite llaves duplicadas (hay que evitarlo)

Tipos de datos

Para la mayoría de tipos de datos, pandas detecta el tipo de datos

- Bool
- Float
- Int

```
Open     985.32
Close    950.44
dtype: float64
```

```
0     4
1     8
2    15
3    16
4    23
5    42
dtype: int64
```

Para otros tipos de datos como strings u objetos complejos, se muestra dtype:object

Se puede especificar el tipo con el parámetro **dtype**

```
0     True
1    False
2    False
dtype: bool
```

Valores faltantes



0



null



undefined

Valores faltantes

- ¿Mary tiene la misma edad de Jaime?
- ¿La edad y el sexo de Jaime es el mismo?

Name	Age	Sex
John	35	M
Mary	NA	F
Jaime	NA	NA

Valores faltantes

- Cuando pandas ve un valor faltante durante la importación de un archivo, la biblioteca sustituye el objeto nan de NumPy.
- El acrónimo nan es la abreviatura de no un número y es un término general para un valor indefinido.
- NaN es un objeto marcador de posición que representa nulidad o ausencia.
- Cuando la serie tiene un nulo, pandas convierte la serie a float

```
import numpy as np

temperatures = [94, 88, np.nan, 91]
pd.Series(data = temperatures)
```

Crear Series a partir de objetos de Python

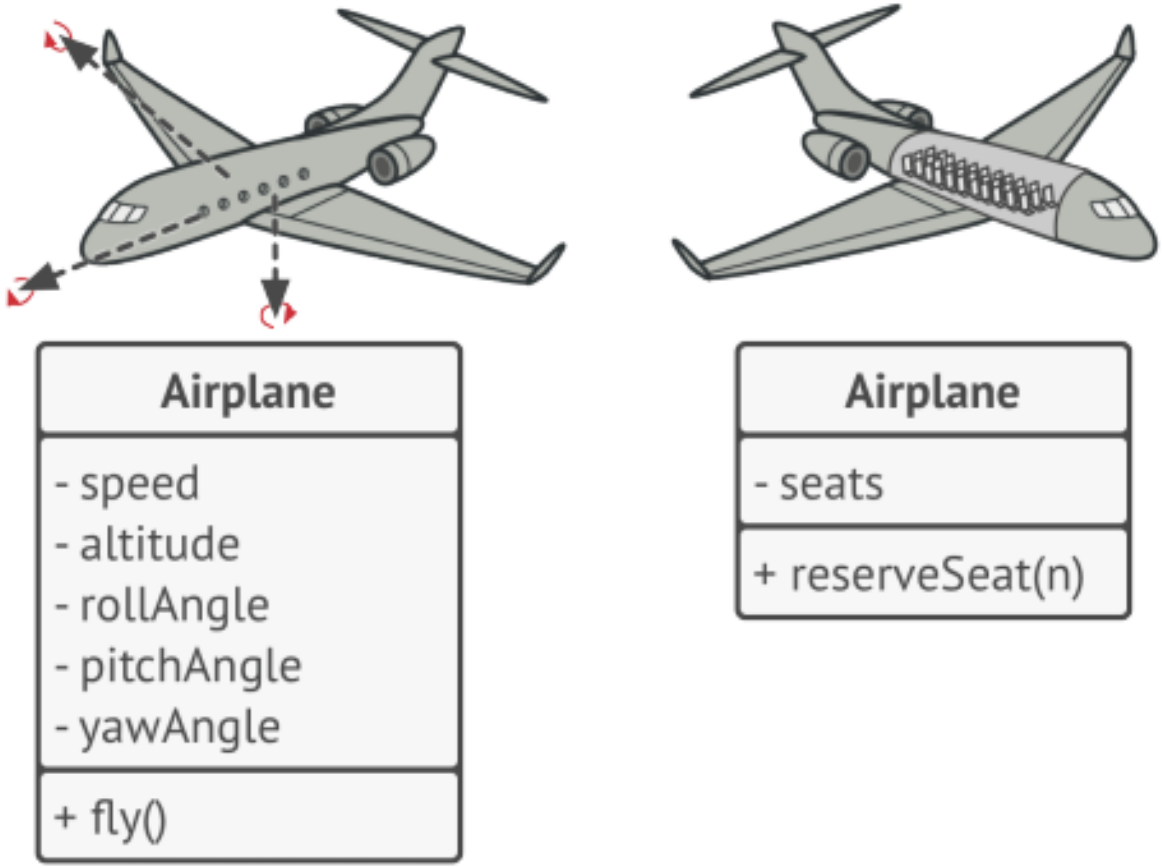
El parámetro data acepta varios tipos de datos

- Diccionarios
- Tuplas
- Conjuntos (transformados a listas)
- Arrays de numpy

Un poco de programación orientada a objetos

El objeto Series

- Atributos
- Métodos



Different models of the same real-world object.

Atributos de las serie

Una serie se compone de objetos más pequeños

- Objeto **Index** de pandas para guardar el índice
- Objeto **ndarray** para guardar los datos

Métodos de las serie

- Una serie tiene funciones/acciones
- Usualmente involucran análisis, cálculos, manipulación de los atributos
- Se identifican por los paréntesis para ingresar parámetros al final de la función (si existen)

Algunas operaciones matemáticas

Operaciones estadísticas

Algunas operaciones matemáticas

- Conteo
- Suma
- Producto
- Suma acumulativa
- Cambio porcentual

- Media
- Mediana
- Desviación estándar
- Máximo
- Mínimo
- Describe
- Muestreo
- Único y conteo de únicos

Operaciones aritméticas

Algunas operaciones matemáticas

- Suma
- Resta
- Multiplicación
- División
- División entera
- Módulo

Broadcasting

Algunas operaciones matemáticas

- Se delegan las operaciones aritméticas a numpy
- Aplica la misma operación a todos los valores de la serie
- Cuando se suman dos series, se suman los valores que tienen el mismo índice
- Cuando se comparan por igualdad (**==**), pandas considera que un **NaN** no es igual a otro **NaN**

Series y funciones integradas de Python

Integración fluída

- A los desarrolladores de Python les gusta respetar ciertos principios de diseño para garantizar la coherencia entre las bases de código.
- Un ejemplo es la integración perfecta entre los objetos de la biblioteca y las funciones integradas de Python. Pandas no es una excepción.
- Podemos pasar una serie a cualquiera de las funciones integradas de Python y obtener un resultado predecible.

PRÁCTICA

Cálculos sobre superheroes

PRÁCTICA

```
superheroes = [
                "Batman",
                "Superman",
                "Spider-Man",
                "Iron Man",
                "Captain America",
                "Wonder Woman"
            ]

strength_levels = (100, 120, 90, 95, 110, 120)
```

1. Use la lista de **superheroes** para completar un nuevo objeto Serie.
2. Use la tupla de **strength_levels** para llenar un nuevo objeto Serie.
3. Cree una serie con los superhéroes como etiquetas de índice y los niveles de fuerza como valores. Asigne la serie a una variable llamada **heroes**.
4. Extraiga las dos primeras filas de la Serie de **heroes**.
5. Extraiga las últimas cuatro filas de la Serie de **heroes**.
6. Determina el número de valores únicos en la serie de **heroes**.
7. Calcula la fuerza promedio de los superhéroes en **heroes**.
8. Calcula las fuerzas máximas y mínimas de los **heroes**.
9. Calcula cuál sería el nivel de fuerza de cada **heroes** si se duplicara.
10. Convierta la Serie de **heroes** en un diccionario de Python.

Gracias por tu asistencia y participación ☺

Contacto

miguela.orjuela@urosario.edu.co

https://www.linkedin.com/in/miguel-orjuela/

https://github.com/maorjuela73

Links de interés

- *https://colab.research.google.com/drive/1RjP5IIvaDgez0MyFTpp8K5rZFvHlfOpB?usp=sharing*